CS 432 Distributed Systems	Lab (2)

Section Content:

- **Java Basic Structure, Sockets, Threads & Concurrency**

Java variable

- Variables must not start with a number. They can only start with an alphabet, underscore (_), or dollar symbol ($).
- A variable name cannot contain symbols like +, -, ., @, #, *, etc.
- Keywords (like int, class, void, etc.) cannot be used as variable names.
- Java is case-sensitive, meaning int abc and int ABC are two different variables.

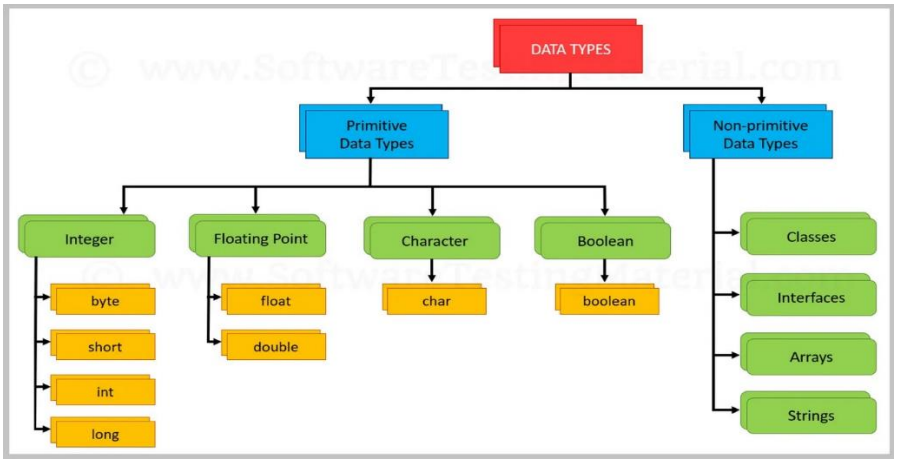
Examples:

- **int abc = 10; // Valid**
- **int abc2 = 20; // Valid**
- **int _abc = 30; // Valid**
- **int $abc = 40; // Valid**
- **int @abc = 50; // Invalid due to the '@' symbol**
- **int abc? = 50; // Invalid due to the '?' symbol**
- **int 72abc = 50; // Invalid**

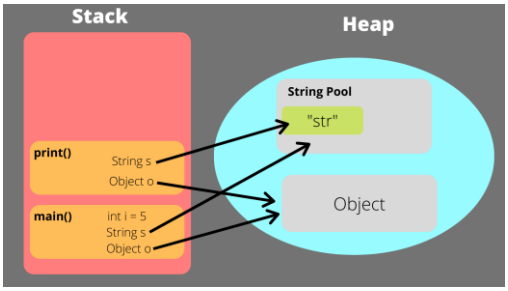
So, Only letters (a-z, A-Z), digits (0-9), the underscore (_), and the dollar sign ($) are allowed in variable names, variable names cannot start with a number.

Java Datatype

In Java, variables are stored in memory based on their data types, and how they are stored depends on whether they are primitive types or non-primitive types.



1. **Primitive Types:** Primitive types are stored directly in memory, and their memory footprint is fixed based on the type. They store the actual data or value.
   - **Where they are stored:** Typically, in the stack memory,
   - **Fixed size in memory**
2. **Non-Primitive Types:** also called **reference types**, store references (or addresses) to objects in memory, not the actual data. The objects themselves are stored in the **heap memory**.
   - **Where they are stored:**
     - **Stack Memory** holds the reference (memory address) to the object.
     - **Heap Memory** stores the actual object.



Java Output

In Java, println(), print(), and printf() are methods used to print output to the console, but they behave slightly differently.

1. println() (Print Line)

- **What it does:** Prints the output and moves the cursor to the next line. It automatically adds a new line after the printed content.
- **Use case:** When you want the output to be on separate lines.
- **Example**

```
System.out.println("Hello");  // Prints "Hello" and moves to the next line
System.out.println("World");  // Prints "World" on a new line
```

Output: Hello
World

2. print() (Print)

- **What it does:** Prints the output but does not move the cursor to the next line. It keeps printing on the same line.
- **Use case:** When you want multiple outputs to be printed on the same line.
- **Example**

```
System.out.print("Hello");  // Prints "Hello" without a newline
System.out.print("World");  // Prints "World" on the same line
```

Output: HelloWorld

3. printf() (Print Formatted)

- **What it does:** Prints formatted output. You can use format specifiers (like %d for integers, %s for strings) to control how the output looks.
- **Use case:** When you want more control over how the output is formatted.
- **Example**

```
int age = 25;
// Uses %d to insert the integer value of age
System.out.printf("I am %d years old", age);
```

Output: I am 25 years old

Java input

Java provides different ways to get input from the user. However, you will learn to get input from user using the object of Scanner class.

The Scanner class of the java.util package is used to read input data from different sources like input streams, users, files, etc.

Using the Scanner Class:

1. Importing the Scanner Class**:**

```
import java.util.Scanner;
```

2. Creating a Scanner Object:

```
Scanner s=new Scanner(System.in);
```

3. Using Scanner Methods:

```
int x=s.nextInt();
```

- Explanation of Methods:
  - ✓ nextByte(): Reads a byte value.
  - ✓ nextShort(): Reads a short value.
  - ✓ nextInt(): Reads an integer value.
  - ✓ nextLong(): Reads a long value.
  - ✓ nextFloat(): Reads a float value.
  - ✓ nextDouble(): Reads a double value.
  - ✓ nextBoolean(): Reads a boolean value (true or false).
  - ✓ nextLine(): Reads a complete line of text, including spaces.
  - ✓ next(): Reads the next word (delimited by whitespace).

Examples

Write a program with java take two number and print its div and reminder

```
package javaapplication1;
import java.util.Scanner;
public class JavaApplication1 {
    public static void main(String[] args) {
        int x,y;
       System.out.print("Please enter num 1: ");
       Scanner s=new Scanner(System.in);
       x=s.nextInt();
       System.out.print("Please enter num 2: ");
       y=s.nextInt();
       System.out.println("DIV : "+x/y+"\n"+"Reminder : "+x%y);
    }
}
```

Write a program with java make swap to two number

```
package javaapplication1;
import java.util.Scanner;
public class JavaApplication1 {
    public static void main(String[] args) {
       int x,y,z;
       System.out.print("Please enter num 1: ");
       Scanner s=new Scanner(System.in);
       x=s.nextInt();     //x=1
       System.out.print("Please enter num 2: ");
       y=s.nextInt();   //y=2
       z=x;             //z=1 ,x=1
       x=y;            //x=2 ,y=2
       y=z;           //y=1 ,z=1
       System.out.println("num 1: "+x+"\n"+"num 2: "+y);
    }
}
```

Write a program with java take number of three digits and print the sum of digits

```
package javaapplication1;
import java.util.Scanner;
public class JavaApplication1 {
    public static void main(String[] args) {
        int x,sum,d;
       System.out.print("Please enter num 1: ");
       Scanner s=new Scanner(System.in);
       x=s.nextInt();
       //System.out.println("sum = "+((x%10)+(x/10)%10+(x/100)));
       d=x%10; // 231%10=1
       sum=d;  // sum=1
       x=x/10; // 231/10=23
       d=x%10; // 23%10=3
       sum+=d; // sum=1+3=4
       x=x/10; // 23/10=2
       sum+=x; // sum=4+2=6
       System.out.println("sum = "+sum);
    }
}
```

Decision-Making statements

As the name suggests, decision-making statements decide which statement to execute and when. Decision-making statements evaluate the Boolean expression and control the program flow depending upon the result of the condition provided. There are two types of decision-making statements in Java, i.e., If statement and switch statement.

If Statement:

In Java, the "if" statement is used to evaluate a condition. The control of the program is diverted depending upon the specific condition. The condition of the If statement gives a Boolean value, either true or false.

There are four types of if-statements given below

1. **Simple if statement**

   This type of if-statement checks a condition, and if the condition is true, it executes the code block inside the if statement.

```
if (condition) {
    // code to execute if condition is true
}
```

Example

```
int x = 10;
if (x > 5) {
    System.out.println("x is greater than 5");
}
```

2. **if-else statement**

   The if-else statement adds an alternative block of code to execute if the condition is false.

```
if (condition) {
    // code to execute if condition is true
} else {
    // code to execute if condition is false
}
```

Example

```
int x = 3;
if (x > 5) {
    System.out.println("x is greater than 5");
} else {
    System.out.println("x is less than or equal to 5");
}
```

3. **if-else-if ladder:**
   This allows checking multiple conditions. If the first condition is false, it moves to the next one, and so on.

```
if (condition1) {
    // code to execute if condition1 is true
} else if (condition2) {
    // code to execute if condition2 is true
} else {
    // code to execute if none of the conditions is true
}
```

Example

```
int x = 7;
if (x > 10) {
    System.out.println("x is greater than 10");
} else if (x > 5) {
    System.out.println("x is greater than 5 but less than or equal to 10");
} else {
    System.out.println("x is 5 or less");
}
```

4. **Nested if-statement:**
   An if statement inside another if statement is known as a nested if-statement. This is used when multiple conditions need to be checked in a hierarchy.

```
if (condition1) {
    if (condition2) {
        // code to execute if both conditions are true
    }
}
```

Example

```
int x = 8;
int y = 12;
if (x > 5) {
    if (y > 10) {
        System.out.println("x is greater than 5 and y is greater than 10");
    }
}
```

Java switch Statement

The **switch statement** is used to execute one block of code from multiple options based on the value of an expression. It is a cleaner alternative to writing multiple if-else-if statements, especially when dealing with multiple possible values for a variable.

```
switch (expression) {
    case value1:
        // code to execute if expression == value1
        break;
    case value2:
        // code to execute if expression == value2
        break;
    // more cases...
    default:
        // code to execute if none of the above cases match
}
```

- **Expression:** The expression inside the switch statement must evaluate to a primitive data type (int, char, short, byte), String, or enum.
- **Case Labels:** The value of each case must match the type of the switch expression. Once a case matches, the corresponding code is executed until a break statement is encountered.
- **break:** Used to exit the switch statement after executing the matched case. **Without** it, execution will "fall through" to the next case.
- **default:** This is an optional case that is executed if no other case matches the expression.

Example

```
String fruit = "apple";

switch (fruit) {
    case "apple":
        System.out.println("Apples are red or green.");
        break;
    case "banana":
        System.out.println("Bananas are yellow.");
        break;
    case "orange":
        System.out.println("Oranges are orange.");
        break;
    default:
        System.out.println("Unknown fruit.");
}
```

Loop Statement

Java provides several types of loop statements that allow code to be executed repeatedly based on a condition. **The main types of loops are:**

1. Java for Loop:

The for loop is typically used when the number of iterations is known beforehand. It consists of three parts: initialization, condition, and increment/decrement.

```
for (initialization; condition; increment/decrement) {
    // code to be executed
}
```

Example

```
for (int i = 0; i < 5; i++) {
    System.out.println("i = " + i);
}
```

2. Java for-each Loop

The for-each loop (also called the **enhanced for loop**) is used to iterate through elements in collections (like arrays or lists). It is especially useful when you don't need the index of the elements

```
for (type variable : array/collection) {
    // code to be executed
}
```

Example

```
int[] numbers = {1, 2, 3, 4, 5};

for (int num : numbers) {
    System.out.println(num);
}
```

3. Java while Loop

The while loop is used when the number of iterations is not known beforehand. It runs the loop as long as the specified condition is true.

```
while (condition) {
    // code to be executed
}
```

Example

```
int i = 0;

while (i < 5) {
    System.out.println("i = " + i);
    i++; // increment i to avoid an infinite loop
}
```

4. Java do-while Loop

The do-while loop is similar to the while loop, but the key difference is that the **condition is checked after the code is executed**, ensuring that the loop runs at least once.

```
do {
    // code to be executed
} while (condition);
```

Example

```
int i = 0;

do {
    System.out.println("i = " + i);
    i++;
} while (i < 5);
```

Jump statements

In Java allow for more control over the flow of loops. They help in skipping parts of the loop or terminating the loop altogether. The primary jump statements in Java are break and continue.

1. **break Statement:** Terminates the loop entirely and moves control to the statement immediately following the loop.

Example

```
for (int i = 1; i <= 3; i++) {
    if (i == 2) {
        break;  // Terminates the loop when i == 2
    }
    System.out.println("i = " + i);
}
```

Output

```
i = 1
```

2. **continue Statement:** Skips the current iteration and moves to the next iteration of the loop.

Example

```
for (int i = 1; i <= 3; i++) {
    if (i == 2) {
        continue;  // Skip the iteration when i == 2
    }
    System.out.println("i = " + i);
}
```

Output

```
i = 1
i = 3
```

Examples

Write a program with java read a text and count the vowel

```
package javaapplication1;
import java.util.Scanner;
public class JavaApplication1 {
    public static void main(String[] args) {
        Scanner s=new Scanner(System.in);
        System.out.print("Please enter the text: ");
        String text =s.nextLine();
        int vowel=0;
        for (int i = 0; i < text.length(); i++) {
            char ch=text.charAt(i);
            if (ch=='e'||ch=='a'||ch=='i'||ch=='o'||ch=='u') {
                vowel++;
            }
        }
        System.out.println("Vowel = "+vowel);
    }
}
```

Java Program to Find Factorial of a Number

```
package javaapplication1;
```

```
import java.util.Scanner;
public class JavaApplication1 {
    public static void main(String[] args) {
        Scanner sc = new Scanner(System.in);
        System.out.print("Enter a non-negative integer: ");
        int number = sc.nextInt();
        int factorial = 1;
        if (number < 0) {
            System.out.println("Factorial is not defined for negative numbers.");
        } else {
            for (int i = 1; i <= number; i++) {
                factorial *= i;
            }
            System.out.println("Factorial of " + number + " is: " + factorial);
        }
    }
}
```

Java 1D Array

An array is a collection of similar types of data.

- **How to declare an array in Java?**

  To declare an array in Java, specify the data type followed by square brackets ([]), and then the array name.

  Syntax:

  dataType[] arrayName;

  Ex:

```java
int[] numbers; // Declares an array of integers
String[] words; // Declares an array of strings
```

- **How to Initialize Arrays in Java?**

  There are several ways to initialize an array in Java:

  1. Using **new** Keyword

     When you know the size of the array but not the values, use the new keyword to allocate memory for the array.

     Syntax:

```
dataType[] arrayName = new dataType[size];
```

     Ex:

```java
int[] numbers = new int[5]; // Creates an integer array of size 5
```

  2. Using an Array Literal

     When you know the values at the time of declaration, you can initialize the array directly.

     Syntax:

```
dataType[] arrayName = {value1, value2, value3, ...};
```

     Ex:

```java
int[] numbers = {1, 2, 3, 4, 5}; // Initializes an array with specific values
```

  3. Initializing Elements Individually

     You can also initialize each element of the array individually after declaring it.

     Ex:

```java
int[] numbers = new int[3];
numbers[0] = 1;
numbers[1] = 2;
numbers[2] = 3;
```

Java 2D Array

A 2D array in Java is an array of arrays, which creates a table-like structure with rows and columns.

- **How to declare an 2D array in Java?**

  To declare a 2D array, you specify the data type, followed by two sets of square brackets.

  Syntax:

  ```
  dataType[][] arrayName;
  ```

  Ex:

  ```
  int[][] matrix;
  ```

- **How to Initialize Arrays in Java?**

  You can initialize a 2D array in different ways, depending on whether you know the values or just the size.

  1. Using **new** Keyword

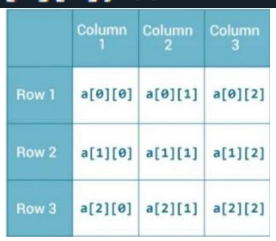
     If you only know the number of rows and columns, you can use the new keyword to create an empty 2D array

     Syntax:

     ```
     dataType[][] arrayName = new dataType[rows][columns];
     ```

     Ex:

     ```
     int[][] matrix = new int[3][3]; // Creates a 3x3 integer 2D array
     ```

  2. Using an Array Literal

     If you already know the values, you can initialize the 2D array directly using an array literal.

     Syntax:

     ```
     dataType[][] arrayName = {
         {value1, value2, ...},
         {value1, value2, ...},
                 ...};
     ```

     Ex:

     ```
     int[][] matrix = {
      {1, 2, 3},
      {4, 5, 6},
      {7, 8, 9}
     };
     ```

3. Initializing Elements Individually

You can initialize each element in the 2D array individually by specifying the row and column indexes.

Ex:

```
int[][] matrix = new int[2][2];
        matrix[0][0] = 1;
        matrix[0][1] = 2;
        matrix[1][0] = 3;
        matrix[1][1] = 4;
```

Java Methods:

A **method** is a block of code that performs a specific task. Methods help organize code, promote reusability, and make programs easier to read and maintain.

- **Declaring a Method in Java**

  A method declaration defines the method's name, return type, parameters, and visibility.

  Syntax:

```
accessModifier returnType methodName(parameterType parameterName, ...) {
                                    // Method body
}
```

  Ex:

```
public int add(int a, int b) {
  return a + b;
}
```

- **Calling a Method**

  To call a method, use the method name followed by arguments (if any) within parentheses.

  Ex:

```
int result = add(5, 3); // Calls the add method and stores the result
```

- **Types of Methods**

  1. **Parameterized Method**

     A method that accepts parameters to perform its task.

     Ex:

```
public void multiply(int x, int y) {
    System.out.println(x * y);
}
```

2. **Non-Parameterized Method**
   A method that does not accept any parameters.
   Ex:

```
public void greet() {
    System.out.println("Hello, world!");
}
```

3. **Return Type Method**
   A method that returns a value to the calling code.
   Ex:

```
public double divide(int a, int b) {
    return (double) a / b;
}
```

4. **Void Method**
   A method with the void return type does not return a value.
   Ex:

```
public void divide(int a, int b) {
    System.out.println( (double) a / b);
}
```

5. **Method Overloading**
   Method overloading allows multiple methods to have the same name but different parameter lists.
   Ex:

```
public int add(int a, int b) {
    return a + b;
}
public int add(int a, int b,int c) {
    return a + b;
}
public int add(double a, double b) {
    return (int)(a + b);
}
```

6. Static Methods

A static method belongs to the class, not an instance. It can be called without creating an object.

Ex 1:

```
class JavaApplication1 {
    public static void main(String[] args) {
        A.display();
    }
}
class A{
    public static void display() {
        System.out.println("This is a static method.");
    }
}
```

Output: This is a static method.

Ex 2:

```
class JavaApplication1 {
    public static void main(String[] args) {
        display();
    }
    public static void display() {
        System.out.println("This is a static method.");
    }
}
```

Output: This is a static method.

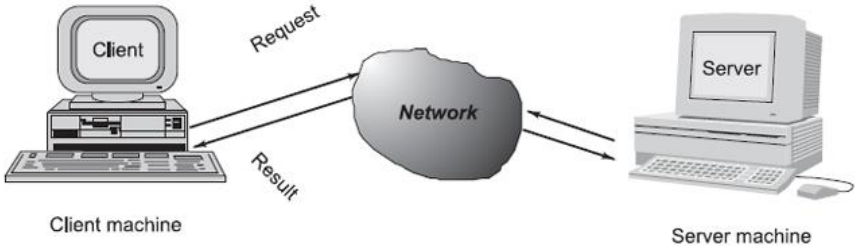
Ex 3:

```
class JavaApplication1 {
    public static void main(String[] args) {
        JavaApplication1 x= new JavaApplication1();
        x.display();
    }
    public void display() {
        System.out.println("This is a static method.");
    }
}
```

Output: This is a non-static method.

Sockets programming

A **socket** is an endpoint used for communication between applications over a network.



To achieve this connection, we need **two fundamental things**:

1. **IP Address (Which computer?):** identifies the computer Address e.g; 192.168.1.10
2. **Port (Which application/service?):** is a logical number used to identify a specific application or service on a compute e.g; `80`.

Imagine a building:

IP Address = Building address

Port = Apartment/room number

If you want to deliver something:

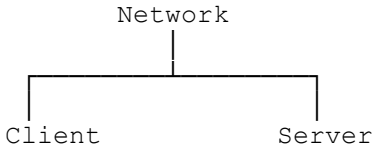
Building: 192.168.1.10

Room: 80

How Client and Server use Sockets?

Now connect this to **Distributed Systems**.

Suppose we have:



The server creates a socket and listens on a port:

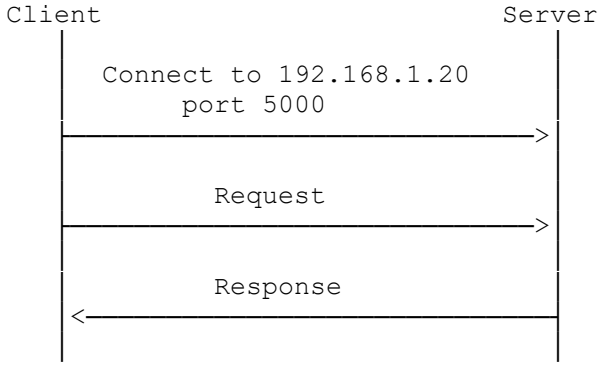
Server

IP = 192.168.1.20

Port = 5000

So the server is listening at: 192.168.1.20:5000

The client wants to communicate with it.



Program:

Here is a simple program that demonstrates Interprocess Communication (IPC) using Message Passing over TCP Sockets. It consists of two independent programs:

- **Node A (Server): Waits for incoming connections and receives a message.**
- **Node B (Client): Connects to the server and sends a message.**

Implementing a Server (Node A)

1. Open the Server Socket
2. Wait for the Client Request
3. Create I/O streams for communicating to the client
4. Close sockets

Code:

```
package nodeaserver;
import java.io.*;
import java.net.*;
public class NodeAServer {
    public static void main(String[] args) {
        try{
            // Create a server socket on port 9000
            ServerSocket server = new ServerSocket(9000);
            System.out.println("Node A is waiting for a connection...");
            // Wait for a client to connect
            Socket connection = server.accept();
            // Read message from the client
            BufferedReader in = new BufferedReader(new
InputStreamReader(connection.getInputStream()));
            String message = in.readLine();
            System.out.println("Received: " + message);
```

```
            // Close connection
            connection.close();
            server.close();
        }catch(Exception e)
        {
            e.printStackTrace();
        }
    }
}
```

Implementing a Client (Node B)

1. Create a Socket Object
2. Create I/O streams for communicating with the server.
3. Close the socket

Code:

```
package nodebclient;
import java.io.*;
import java.net.*;
public class NodeBClient {
    public static void main(String[] args) {
        try{
            // connect to NodeA
            Socket socket = new Socket("localhost",9000);
            // send message to server
            PrintWriter out= new PrintWriter(socket.getOutputStream(), true);
            out.println("Hello from Node B!");
            System.out.println("Message sent to node A");
            // Close connection
            socket.close();
        }catch(Exception e)
        {
            e.printStackTrace();
        }
    }
}
```

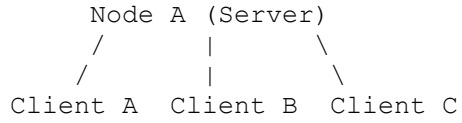
Output:

```
Node A is waiting for a connection...
Received: Hello from Node B!
BUILD SUCCESSFUL (total time: 8 seconds)

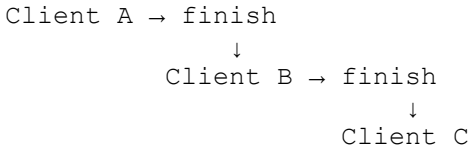
run:
Message sent to node A
BUILD SUCCESSFUL (total time: 0 seconds)
```

Threads

Node A is a server and receives requests from many clients:



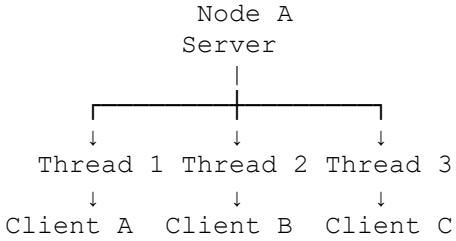
If Node A uses one thread**:**



Each client must wait.

Using Threads

Node A can create a separate thread for each client**:**



Now the server can handle multiple clients concurrently**.**

Concurrency

Concurrency means multiple clients/tasks are being handled during the same period of time.

For example:

Thread 1 → Client A
Thread 2 → Client B
Thread 3 → Client C

This is very common in distributed systems because a server may receive many requests at the same time.

Program:

Concurrent Execution in a Distributed System This program demonstrates the Concurrent Execution characteristic of a distributed system. Two independent nodes, Node-A and Node-B, perform their tasks simultaneously using separate threads. The main program waits for both threads to complete before displaying a final message.

Code

```
package concurrentexecutiondemo;
public class ConcurrentExecutionDemo {
   public static void main(String[] args) {
     // Node-A task
     Thread nodeA = new Thread(() -> {
       for (int i = 1; i <= 3; i++) {
       System.out.println("Node-A processing task " + i);
     }
     });
     // Node-B task
     Thread nodeB = new Thread(() -> {
      for (int i = 1; i <= 3; i++) {
          System.out.println("Node-B processing task " + i);
       }
      });
     // Start both nodes concurrently
       nodeA.start();
       nodeB.start();
     // Wait for both nodes to finish
     try {
       nodeA.join();
       nodeB.join();
     }
     catch (InterruptedException e) {
        e.printStackTrace();
     }
     // Final message
     System.out.println("All nodes have completed their work.");
   }
}
```

Output

```
Node-B processing task 1
Node-B processing task 2
Node-A processing task 1
Node-B processing task 3
Node-A processing task 2
Node-A processing task 3
All nodes have completed their work.
BUILD SUCCESSFUL (total time: 0 seconds)
```